
Quantum Information

Verification, Optimization, and Transformation of
Quantum Programs

Design and implementation of a quantum simulator and a
corresponding approximation

Project Report
David Ahmad Zonneveld Michel

Loria Research Lab
July 31, 2017

*Supervisors : Simon Perdrix
Emmanuel Jeandel*

Abstract

The quantum world is unlike anything we've seen before. Particles can be in a superposition of states. They can be correlated in such a way that the manipulation of one affects another one.

While these new properties make the prospect of quantum computers very promising, they also make the design, implementation, verification, and optimization of quantum programs in a classical world very challenging.

By using formal methods, I designed and implemented a quantum simulator for a classical machine. I suggested an approximation for dealing with the natural exponential blowup arising from the simulation of quantum in classical machines, and provided a concrete implementation thereof. The mathematical analysis of the approximation is also provided. One thing that makes this approximation so appealing is that it is extremely easy to choose how much precision one is willing to give up in exchange of efficiency.

The simulator, together with the approximation, allows us both to analyze quantum programs with default sets of quantum operators, and to add custom defined ones. It provides anybody interested in the quantum area and without access to a universal quantum machine with a means to verify the correctness of quantum algorithms, help in the design of new ones, find gates equivalences, and demonstrate properties of existing quantum algorithms, just to name a few.

Contents

List of Figures	3
1 Introduction	4
2 Background Information	7
2.1 Quantum Information Processing	7
2.1.1 Qubit Status	7
2.1.2 Multiple-qubit Status	7
2.1.3 Quantum Measurements	8
2.1.4 Probabilistic Status	8
2.1.5 Quantum Gates	8
2.1.6 Quantum Program	9
2.2 Formal Methods	9
2.2.1 Backus Naur Form	9
2.2.2 Canonicalization	9
3 Design and Implementation	11
3.1 Static Representation	11
3.1.1 Pauli Matrices	11
3.1.2 Backus Naur Form Representation	11
3.1.3 Canonical Form	12
3.1.4 Canonicalization	13
3.2 Dynamic Representation	14
3.2.1 Syntactic	15
3.2.2 Semantics	15
3.3 Simulator run	18
4 The ϵ Approximation	20
4.1 Description	20

Contents	2
4.2 Analysis	20
Acknowledgements	23
Bibliography	24
A Termination demonstration	25
A.1 Intuitive demonstration	25
A.2 Mathematical demonstration	26

List of Figures

1.1	Von Neumann's density matrix status representation.	5
2.1	Probabilistic vs deterministic representations of Quantum Measurements	8
2.2	The term rewriting algorithm.	10
3.1	The Backus-Naur Form representation of a quantum status. .	12
3.2	Canonicalization transformation 1: Multiplication by a scalar	13
3.3	Canonicalization transformation 2: Tensor product distribution over addition	13
3.4	Canonicalization transformation 3: Multiplication distribution over addition	14
3.5	Canonicalization transformation 4: Combination of similar terms	14
3.6	Default gates	18
4.1	The ϵ approximation	21

Chapter 1

Introduction

Quantum computers are still in their infancy. They are what we would call "proof of concept". Until high scale universal quantum computers exist, quantum programs need to be designed, manipulated, optimized, and tested using classical computers. This is far from a trivial problem.

The problem being the discrepancy between the two worlds. Using the density matrix formalism, the status of programs using n qubits of data (the quantum basic unit of information) would require $2^n \times 2^n$ classical bits to be represented (Von Neumann 1927) (see figure 1.1).

As a matter of fact, Feynman demonstrated already in 1981 that this "exponential blowup", as he called it, was, for a complete simulation of all quantum phenomena in a classical machine, unavoidable. (Feynman 1981)

One way of dealing with this exponential blowup is through the use of approximations – nby representing the exact quantum system but an approximation thereof.

During this internship I:

- Used formal methods and dynamic programming techniques for the implementation of a quantum simulator.
- Suggested an approximation (the ϵ approximation) for dealing with the exponential size of the status of the simulator.
- Provided an analysis and concrete implementation of such an approximation.

This report is organized as follows: In Chapter 2 we briefly describe the minimum background needed for the rest of the report. Section 2.1 is really

Figure 1.1: The evolution of a three qubit quantum system using Von Neumann’s density matrix status representation.

just a very shallow view of the beauties of quantum information processing. Only the concepts strictly necessary for understanding the quantum simulator are included. For the interested reader, a more thorough introduction, like Nielsen & Chuang (2010), is recommended. Section 2.2 describes the formal techniques used in the implementation of the simulator.

Chapter 3 goes through the design and implementation of the simulator itself. Section 3.1 describes the representation of a quantum status in our simulator, together with its canonicalization, or standardization, for combining similar terms, making the dynamic evolution easier, and providing us a way to approximate the status. Section 3.2 describes the dynamics of the simulator. Section 3.3 explains how a full run of a quantum algorithm over a given initial status works.

Chapter 4 describes the approximation suggested and provides an analysis of its complexity.

Chapter 2

Background Information

2.1 Quantum Information Processing

The concepts discussed in this section are taken from (Nielsen & Chuang 2010).

Quantum information processing is the abstraction of physical phenomena into mathematical terms. This abstraction, just as abstracting a physical computer into chains of 1s and 0s, is independent of the specific implementation.

2.1.1 Qubit Status

The basic piece of information in quantum information processing is the qubit. A qubit has an infinite amount of possible status. A qubit can be 0, it can be 1, or it can be in a superposition of both (a bit of 0, a bit of 1). In mathematical terms, we say that the status of a qubit is given by an element of the 2-Hilbert Space $\mathbb{C}^2$. The first complex number representing the amplitude of 0, the second the amplitude of 1.

2.1.2 Multiple-qubit Status

A Multiple-qubit system can be in a superposition of all of its possible quantum states. For an n -qubits system, this can be represented as an element of the 2^n complex Hilbert Space ($\mathbb{C}^{2^n}$).

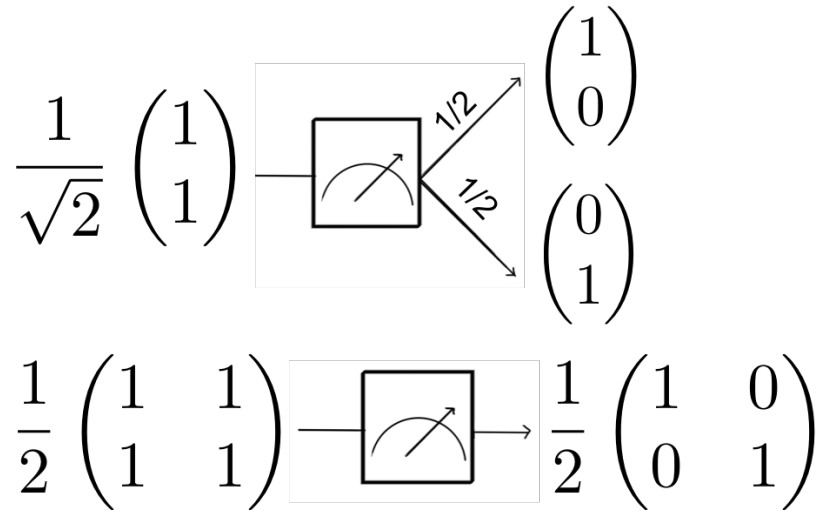


Figure 2.1: A quantum measurement. It can either be seen as a probabilistic evolution of a deterministic representation of the quantum system or as a deterministic evolution of a probabilistic representation of the quantum system

2.1.3 Quantum Measurements

A quantum measurement is probabilistic. After performing a measurement in one qubit, the system probabilistically collapses into one or another quantum statuses. In other words, after measuring the system, it will either be in one ($\mathbb{C}^{2^n}$) status or in another ($\mathbb{C}^{2^n}$) status.

2.1.4 Probabilistic Status

One way of representing this probabilities is by using a matrix instead of a vector. We used the density matrix formalism introduced in Von Neumann (1927). With this formalism, the probabilities are embedded into the status representation itself (See figure 2.1)

2.1.5 Quantum Gates

A quantum gate is a circuit that transforms one valid quantum status into another one. Using the density matrix status representation, it can be seen as a linear transformation from $\mathbb{C}^{2^n \times 2^n}$ to $\mathbb{C}^{2^n \times 2^n}$

2.1.6 Quantum Program

A quantum program can be seen as the evolution of quantum system statuses through the application of quantum measurements and quantum gates.

2.2 Formal Methods

2.2.1 Backus Naur Form

One way to abstractly describe context-free grammars (which include the syntax of quantum statuses and the syntax of quantum programs) is by using the Backus-Naur-Form (Naur 1981). A Backus-Naur-Form description is simply a set of derivation rules with the following format: $\langle \textit{Nonterminal} - \textit{Symbol} \rangle ::= \textit{Expression} | \dots | \textit{Expression}$ where " $::=$ " means "is defined as" and "|" means "or". These rules give the valid replacements for the symbols in the left. The symbols in the left are defined, and thus can be replaced, by any of the expressions in the right. By combining many of these descriptions very complex symbols (including symbols for representing valid quantum programs and valid quantum statuses) can be described.

2.2.2 Canonicalization

One way of achieving canonicalization (having terms in a canonical, or standard, form) is through the use of rewriting systems.

Term rewriting systems

As described in Book & Otto (1993), a reduction system (an alternate term for rewriting systems) is a set of objects together with a reduction relation over that set. The reduction relation can be seen as a set of rewriting rules with the following format:

$$L \rightarrow R$$

Meaning that the term in the left is equivalent, and can be reduced (or rewritten) as the term on the right.

The rewriting algorithm

Given a term rewriting system, as described in 2.2.2, together with an input object from the set for which the reduction rules are defined, the object is

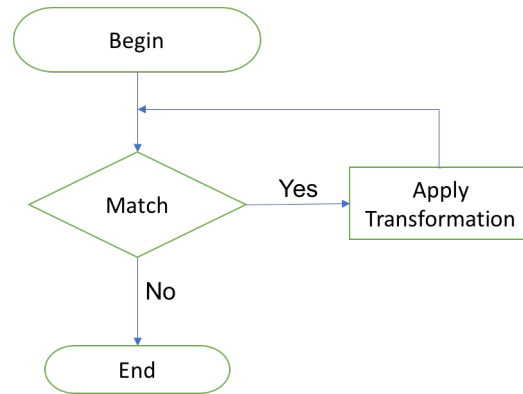


Figure 2.2: The term rewriting algorithm.

canonicalized, or standardized, by applying the following simple algorithm:

```

while Any of the rule matches any subterm of the term being simplified
do
    simplify that subterm through the application of that rule.
end while
  
```

Where:

- A subterm is any term (possibly the term itself) within a term.
- A rule is said to match a subterm if there exists a variable substitution such that by applying that variable substitution to its left-hand side it becomes equal to that subterm.
- Applying a simplification to a subterm means replacing the subterm by the right-hand side of the rule used in the matching after applying the substitution used in the matching.

Naturally, the loop can be infinite. Thus, for a term-rewriting system a termination proof is required to demonstrate that we reach the canonical form in a finite amount of time. Many ways of doing this exist. One of them is demonstrating that a given abstract quantity always reduces when going from left to right by, for example, finding a polynomial interpretation where this is the case. (Lankford 1975. Canonical inference).

Chapter 3

Design and Implementation

3.1 Static Representation

The first question to answer is how a quantum state is going to be represented. The density matrices explained in the background and described in Von Neumann (1927) can be used to fully represent the probabilistic, superposed nature of any quantum system, so that will be our starting point.

3.1.1 Pauli Matrices

It turns out that all valid quantum states can be seen as combinations of four simpler matrices (Pauli 1927), namely:

$$X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

known as the Pauli Matrices Π . Then, the status representation of an n -qubit machine can be seen as a linear combination of the n th Pauli group (Π^n) defined as follows:

$$\Pi^n = \{e^{i\phi} A_1 \otimes \dots \otimes A_n : \forall j \in \{1, \dots, n\} A_j \in \{I, X, Y, Z\}, \phi \in \{0, \frac{\pi}{2}, \pi, \frac{3}{2}\pi\}\}$$

This is the representation that we'll be using in our simulator.

3.1.2 Backus Naur Form Representation

In order to make the representation explained in the previous section understandable to computers, I use the Backus-Naur representation (Naur 1981),

$$\begin{aligned}
& \textit{Status} ::= \{ \textit{ExpressionList} \} \\
& \textit{ExpressionList} ::= \textit{Expression} \\
& \quad | \textit{Expression}, \textit{ExpressionList} \\
& \textit{Expression} ::= \textit{Number} \\
& \quad | \textit{StateIdentifier} \\
& \quad | \textit{Expression} / \textit{Expression} \\
& \quad | \textit{Expression} * \textit{Expression} \\
& \quad | \textit{Expression} + \textit{Expression} \\
& \quad | \textit{Expression} - \textit{Expression}
\end{aligned}$$

Figure 3.1: The Backus-Naur Form representation of a quantum status. "," was used to represent the Tensor Product ($\otimes$) because of its keyboard-input simplicity. "Number" is any complex number. A library was used for keeping the representation symbolic (ex/ keeping " $\sqrt{2}$ " as " $\sqrt{2}$ " instead of "1.414...").

used to describe context-free grammars; which in turn describe context-free languages, like the one formed by mathematical description given in the previous section. Using this description we can represent a quantum state as shown in figure 3.1.

3.1.3 Canonical Form

In order for us to be able to easily compare and combine similar terms, a canonical form was needed. For us, a quantum status in canonical form is a linear combination of tensor products. That is, a tree of size four where:

1. On the root we have the addition operator
2. On the second level we have multiplication operators
3. On the third level we have the complex scalar in symbolic form and the tensor product operators
4. On the fourth level we have the Pauli Matrices as the leaves of our tree.

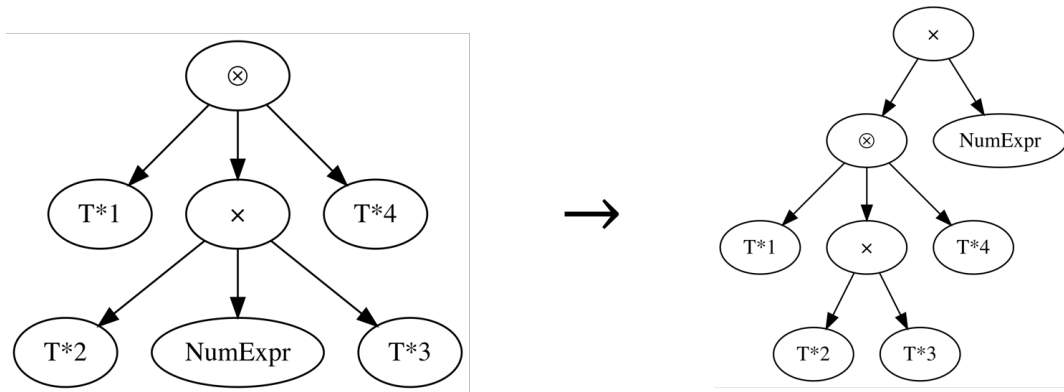


Figure 3.2: Canonicalization transformation 1: Multiplication by a scalar

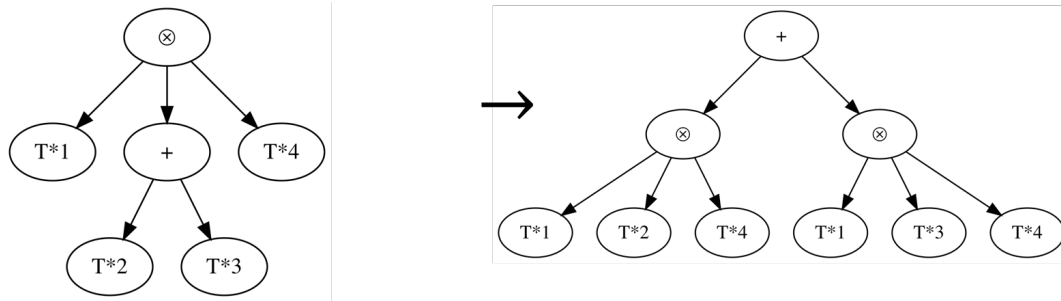


Figure 3.3: Canonicalization transformation 2: Tensor product distribution over addition

3.1.4 Canonicalization

In order to achieve the canonicalization described in the previous section it suffices to define a set of rewrite rules and apply a term rewriting algorithm, as described in 2.2.2.

Let "W", "X", "Y", and "Z" be any matrix terms (i.e. they could stand for any full matrix expression) and "k" any complex number. The rewriting rules used are as follows:

- **Scalar Multiplication.** We can always take a scalar out of the tensor product (figure 3.2):

$$X \otimes kY \otimes Z \rightarrow k(X \otimes Y \otimes Z)$$
- **Tensor distribution over addition.** Tensor product is distributed over additions (figure 3.3):

$$X \otimes (Y + Z) \otimes W \rightarrow X \otimes Y \otimes W + X \otimes Z \otimes W$$
- **Multiplication distribution over addition.** Multiplication of matrices

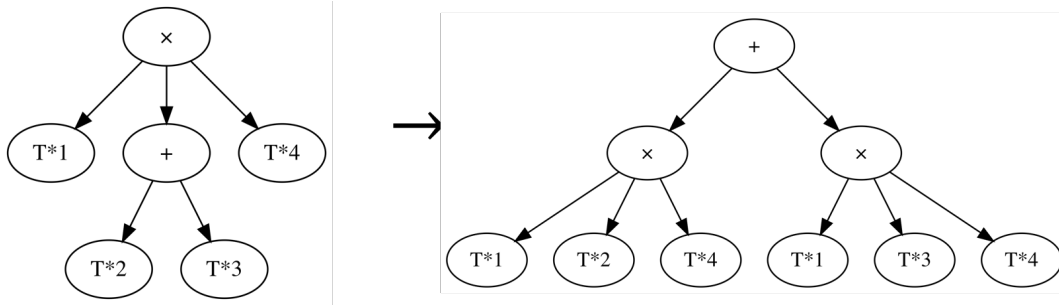


Figure 3.4: Canonicalization transformation 3: Multiplication distribution over addition

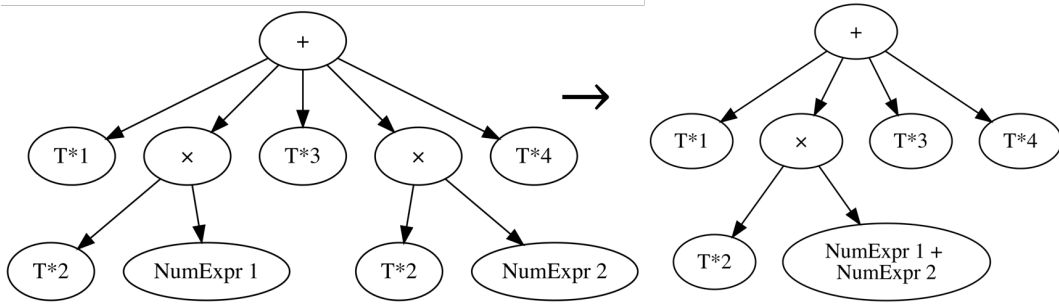


Figure 3.5: Canonicalization transformation 4: Combination of similar terms

can be distributed over addition of matrices (figure 3.4):

$$X \times (Y + Z) \times W \rightarrow X \times Y \times W + X \times Z \times W$$

- **Combining similar terms** If after reaching the normal form we have the addition of two identical matrix terms with different scalars, we can just combine them by adding the scalars together (figure 3.5):

$$X + kY + Z + k2Y + W \rightarrow X + (k + k2)Y + Z + W$$

3.2 Dynamic Representation

Until now we have worried about the static representation of a quantum status; about how the quantum system would look like at a given point. Let us now concentrate on the dynamics.

A quantum status changes when we pass it through a quantum circuit (gates) or, because this is quantum, by simply observing (measuring) the system (Heisenberg 1930). While the former changes the system in a deterministic way, the latter does it in a probabilistic way. However, since we are already considering the probabilities by using probability matrices in-

stead of vectors; we can see both as deterministic transformations over the probabilistic state representation. (Von Neumann 1927).

3.2.1 Syntactic

For the quantum program a very slight modification of the syntax described in Perdrix (2008) was used. A quantum program can be seen as a sequence of commands. A command can either be the skip command, a command followed by another command, an if-then-else block, or a gate application:

$$\begin{aligned}
 C &::= \text{skip} \\
 &| C_1; C_2 \\
 &| \text{if } q\text{QubitNumber} \text{ then } C_1 \text{ else } C_2 \\
 &| \text{Gate}(\text{QubitArguments}) \\
 \text{QubitArguments} &::= \text{QubitNumber} \\
 &| \text{QubitNumber}, \text{QubitArguments}
 \end{aligned}$$

3.2.2 Semantics

Quantum Gate Semantics

Since quantum gates are linear (Nielsen & Chuang 2010) and after canonicalization we already have the status represented as a linear combination of matrix terms, we can simply apply the gates to each of the individual terms in turn (Distribute over addition, pass through scalars). That is, we just ignore the first two levels of the tree status representation described in the canonicalization section, and apply the transformations to the third level.

Now, since each of the individual terms is a tensor product of Pauli matrices and identity, it suffices to apply the transformation to the corresponding subterm[s] as specified in the gate parameters. For example, if the input is "T(2)", at the third level of our tree we simply apply the "T" gate to its second subterm.

Now we get to the Pauli matrices level. To apply the gate over a Pauli Matrix we use a dynamic programming technique known as memoization (Norvig 1991). That is, we calculate the result of the application of the gate over each of the Pauli matrices and identity (or combination of gates if its a multiple-qubit gate), save the result in memory once, for using it many times, and, whenever needed, recover the result from memory.

The only piece of information that is missing is how to apply a quantum gate over each of the Pauli Matrices and Identity. This is easy, it's just applying quantum mechanical principles as explained for example in Nielsen & Chuang (2010). We simply multiply the matrix by the gate on the left and by the gate transposed complex conjugate on the right. Finally, for this implementation, we compare it with the combinations of our original Pauli matrices and identity to see which of these matrices are left after the transformation.

For example, for calculating the semantics of applying gate T over the Pauli Matrix "X" ($T(X)$) we would proceed as follows:

$$\begin{aligned}
 T(X) &= T \times X \times T^\dagger \\
 &= \begin{pmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{pmatrix} \times \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \times \begin{pmatrix} 1 & 0 \\ 0 & e^{i(-\frac{\pi}{4})} \end{pmatrix} \\
 &= \begin{pmatrix} 0 & e^{i(-\frac{\pi}{4})} \\ e^{i\frac{\pi}{4}} & 0 \end{pmatrix} \\
 &= \frac{X + Y}{\sqrt{2}}
 \end{aligned}$$

After repeating this procedure for each of the three Pauli Matrices and identity we obtain the following semantics for gate T:

$$[[T]] = \lambda S. \lambda Q. (S[X_Q := \frac{X + Y}{\sqrt{2}}, Y_Q := \frac{Y - X}{\sqrt{2}}])$$

This is the result saved in memory for quick use during the transformations described in the first paragraphs.

Quantum Measurement Semantics

As discussed above, quantum measurements can be seen as either probabilistic linear transformations on deterministic representation of the quantum state or as deterministic linear transformations over probabilistic representations thereof. The status representation we chose is already probabilistic, so our measurements are deterministic.

Since they are also linear, they also distribute over addition (first level of the canonical form representation) and pass through scalars (second level of the canonical form representation quantum) as discussed in the Quantum Gate Semantics Section.

Then, in the Pauli tensor-product level, we just select the appropriate subterm depending on the qubit that we want to measure.

Finally, for the application of the measurement itself, we use the definition explained in for example Nielsen & Chuang (2010). We just apply one pair of operators depending on the basis that we are working with. For the computational basis, the operators are $|0\rangle\langle 0|$ and $|1\rangle\langle 1|$. By applying this operator to each of the Pauli matrices and Identity and calling the $|0\rangle\langle 0|$ operator "True" and the $|1\rangle\langle 1|$ operator "False" the following semantics for the measurements on the computational basis for the Pauli matrices and identity was obtained:

$$\begin{aligned} \llbracket \text{True Measurement} \rrbracket &= \lambda S. \lambda Q. (S[I_Q := \frac{I+Z}{2}, \\ &\quad X_Q := \epsilon, \\ &\quad Y_Q := \epsilon, \\ &\quad Z_Q := \frac{I+Z}{2}]) \\ \llbracket \text{False Measurement} \rrbracket &= \lambda S. \lambda Q. (S[I_Q := \frac{I-Z}{2}, \\ &\quad X_Q := \epsilon, \\ &\quad Y_Q := \epsilon, \\ &\quad Z_Q := \frac{Z-I}{2}]) \end{aligned}$$

If-then-else block Semantics

We use the if semantics described in Perdrix (2008). Namely, the semantics of an if-then-else block is simply a True Measurement followed by the application of the then commands or a False Measurement followed by the application of the else commands. Again, in an actual quantum computer which one of them is triggered is probabilistic. Here, we're already considering the probabilities by the use of probability matrices. As described in Von Neumann (1927), the resulting state after a measurement is given by the addition of the two probability matrices. Therefore, the result of an if-then-else block is given by the result of a true measurement, followed by the then commands, added to the result of a false measurement, followed

$$\begin{aligned}
X &= \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \\
H &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\frac{\pi}{4}} \end{pmatrix} CNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}
\end{aligned}$$

Figure 3.6: Default gates used in the simulator. this gates, one possible set of gates taken as universal gates (Meaning that all valid quantum gates can be approximated by them) are provided by default. No gate definition is needed if only using gates in this set.

by the else commands:

$$\begin{aligned}
\llbracket \text{if } Q \text{ then } C_1 \text{ else } C_2 \rrbracket &= \lambda S. (\llbracket C_1 \rrbracket (\llbracket \text{True Measurement} \rrbracket (S, Q)) \\
&\quad + \llbracket C_2 \rrbracket (\llbracket \text{False Measurement} \rrbracket (S, Q)))
\end{aligned}$$

Semantic, others

The only two commands left to explain are skip and composition. Again, the semantics described in Perdrix (2008) were used. Skip leaves the quantum state unchanged and Composition applies first the command on the left then the command on the right:

$$\begin{aligned}
\llbracket \text{skip} \rrbracket &= \mathbb{I} \\
\llbracket C_1; C_2 \rrbracket &= C_1 \circ C_2
\end{aligned}$$

3.3 Simulator run

The input of our simulator is:

1. An initial status with a syntax like the one described in section 3.1.2
2. A program with the syntax described in section 3.2.1
3. The gates to be used together with its definitions. By default (without the need of providing its definitions) the gates in the universal set shown in figure 3.6 will be accepted.

4. A value for ϵ (used for the approximation, explained in more detail in section 4.1.)

And a run of the simulator works as follows:

1. Transform the string representation of the status into a tree representation thereof.
2. Transform the string representation of the program into a tree representation thereof.
3. Canonicalize the initial status.
4. If non-standard gates given, calculate and memoize their semantics for each of the Pauli matrices.
5. For each command in the program:
 - (a) Execute the command over the canonicalized representation of the status.
 - (b) Canonicalize the resulting status after the execution of the command.
 - (c) Combine similar terms.
 - (d) Apply the approximation to reduce the size of the status (section 4.1).

Chapter 4

The ϵ Approximation

For dealing with the exponential blow up on the number of qubits described in Feynman (1981), an approximation was suggested. This approximation was called the ϵ Approximation.

4.1 Description

The ϵ approximation works as follows: Since after canonicalization (section 3.1.4 we already have the status represented as a linear combination of tensor products of Pauli matrices, we can simply diminish the size of the status by only keeping the most significant terms. How to determine which terms are significant and which are not? It is given by the user, depending on:

The amount of resources available.

The precision required.

We call this value ϵ . We keep all terms whose absolute value are at least ϵ , and ignore all others (see figure 4.1 for an example).

4.2 Analysis

With one qubit, after canonicalization and combining similar terms (section 3.1.4), the possible statuses are linear combinations of X , Y , Z , I matrices. Similarly, for two qubits, the possible statuses are linear combinations of $X \otimes X$, $X \otimes Y$, $\dots$, $I \otimes I$ matrices.

$$\begin{aligned}
InitialState &= \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} = \frac{Z+I}{2} \otimes \frac{Z+I}{2} \\
&= \frac{1}{4}Z \otimes Z + \frac{1}{4}Z \otimes I + \frac{1}{4}I \otimes Z + \frac{1}{4}I \otimes I \quad \epsilon = \frac{1}{4} \text{ approximation of the status} \\
H(Qubit1) &\rightarrow \frac{1}{4}I \otimes I + \frac{1}{4}I \otimes Z + \frac{1}{4}X \otimes I + \frac{1}{4}X \otimes Z \\
T(Qubit1) &\rightarrow \frac{1}{4}I \otimes I + \frac{1}{4}I \otimes Z + \frac{\sqrt{2}}{8}X \otimes I + \frac{\sqrt{2}}{8}X \otimes Z \frac{\sqrt{2}}{8}Y \otimes I + \frac{\sqrt{2}}{8}Y \otimes Z
\end{aligned}$$

Figure 4.1: The ϵ approximation

Generalizing, for n qubits, without any approximation, after canonicalization and combining similar terms, the possible statuses are linear combinations of the n -tensor-products of those 4 matrices. That is, linear combinations of a 4^n set. ($X \otimes X \otimes \dots \otimes X, X \otimes X \dots \otimes Y, \dots, I \otimes I \otimes \dots \otimes I$).

With the approximation, the maximum amount of terms we could possibly have is when all terms equal epsilon. When will this happen? Well, the worst-case scenario is when the absolute value of the individual terms are reducing the least and the number of terms are expanding the most (more specifically, when the product from term reduction factor to number of terms expansion is the biggest). This happens with gates where the factor of term increase is inc and the absolute value of term reduction is $\sqrt{inc}$, for any inc . When this happens, the amount of terms increases with the square of the inverse of the reduction factor. Therefore, when we reach ϵ , we'll have at a maximum $\left\lfloor \frac{1}{\epsilon^2} \right\rfloor$ terms.

So, the maximum size of our status is now $O(\min(4^n, \frac{1}{\epsilon^2}))$, where n is the number of qubits and ϵ is an input parameter! Proportional to the inverse of the square of the input parameter ϵ . So, with our approximation, it's up to the user to decide how much precision he is willing to give up in exchange of increased efficiency in order to partially solve the exponential blowup that would occur if using an exact simulation.

Conclusions

During this internship I used formal methods in the design of a simulator, implemented the simulator, suggested an approximation for dealing with the exponential blowup discussed in Feynman (1981) when simulating quantum machines in the classical world, and implemented such an approximation.

While we don't have access yet to a big-enough universal quantum computer, there are many cases where a quantum simulator would be extremely useful: It can be used to verify the correctness of existing algorithms, to help in the development of new ones, to find gate equivalences, and to demonstrate properties of quantum algorithms, among other things. Undoubtedly, this simulator, together with its underlying approximation, will be a very useful tool for anybody interested in delving more deeply into the development, verification, and optimization of quantum algorithms.

Acknowledgements



With the support of the Dependable Software Systems (DESEM) Programme of the European Union.

Special thanks to my supervisors, who guided me into my deepening into the quantum world.

Bibliography

- Book, R. V. & Otto, F. (1993), *String-rewriting Systems*, Springer Verlag, pp. 31–64.
- Feynman, R. (1981), ‘Simulating physics with computers’, *International Journal of Theoretical Physics* .
- Heisenberg, W. (1930), *Die physikalischen Prinzipien der Quantentheorie.*, Hirzel S. Verlag.
- Lankford, D. S. (1975. Canonical inference), *Memo ATP-32, Automatic Theorem Proving Project*, University of Texas.
- Naur, P. (1981), Report on the algorithmic language algol 60, Technical report.
- Nielsen, M. A. & Chuang, I. L. (2010), *Quantum Computation and Quantum Information*.
- Norvig, P. (1991), ‘Techniques for automatic memoization with applications to context-free parsing’, *Comput. Linguist.* **17**(1), 91–98.
- Pauli, W. (1927), ‘Zur quantenmechanik des magnetischen elektrons’, magazine.
- Perdrix, S. (2008), ‘Quantum entanglement analysis based on abstract implementation’.
- Von Neumann, J. (1927), Wahrscheinlichkeitstheoretischer aufbau der quantenmechanik.

Appendix A

Termination demonstration

The term-rewriting system, described in 3.1.4 and provided here for convenience, used for the canonicalization of quantum statuses was the following:

$$\begin{aligned} X \otimes kY \otimes Z &\rightarrow k(X \otimes Y \otimes Z) \\ X \otimes (Y + Z) \otimes W &\rightarrow X \otimes Y \otimes W + X \otimes Z \otimes W \\ X \times (Y + Z) \times W &\rightarrow X \times Y \times W + X \times Z \times W \\ X + kY + Z + k2Y + W &\rightarrow X + (k + k2)Y + Z + W \end{aligned}$$

A proof of its termination is provided.

A.1 Intuitive demonstration

All our canonicalization algorithm does is transforming quantum statuses into linear combinations of multiplications of scalars by tensor products of Pauli Matrices, followed by the combination of similar terms. In other words, it "pushes" all additions to the root level of our tree representation of quantum statuses, multiplications to the second level, and tensor products to the third level. Then, if similar terms are encountered it combines them.

This "pushing" is of course finite, as a time must come when everything is pushed to their place (everything is ordered) and no more pushing can be done. As for the combination of similar terms, it doesn't change the ordering already established and decreases the branches of our tree, thus a time must also come when reducing the tree is no longer possible and the rewriting terminates.

A.2 Mathematical demonstration

As explained in section 2.2.2, for the mathematical demonstration we demonstrate that for all rules an abstract quantity diminished when going from left to right. We use the method of polynomial interpretations, due to Lankford (1975. Canonical inference) for this purpose. We use the following (monotonic for the non-negative integers) polynomial interpretations:

$$\begin{aligned}
 \text{Tensor Product :} \quad & \tau(\alpha \otimes \beta) = \tau(\alpha)^2 \cdot \tau(\beta) \\
 \text{Matrix - Matrix Multiplication :} \quad & \tau(\alpha \times \beta) = \tau(\alpha) \cdot \tau(\beta) + \tau(\alpha) \\
 \text{Scalar - Matrix Multiplication :} \quad & \tau(\alpha\beta) = \tau(\alpha) \cdot \tau(\beta) + 1 \\
 \text{Matrix Addition :} \quad & \tau(\alpha + \beta) = \tau(\alpha) + \tau(\beta) \\
 & \tau(0) = 2 \\
 & \tau(1) = 2
 \end{aligned}$$

Then, to prove termination, it suffices to prove that each rule is reducing. That is, that the left of each rule is greater than the right of it under any non-negative interpretations. Equivalently, for each rule we need to demonstrate that $\tau(l) - \tau(r) > 0$ for any non-negative integer interpretation. Let's do that.

For the first rewriting rule we have:

$$\begin{aligned}
 \tau(l) &= \tau(X \otimes kY \otimes Z) \\
 &= (\tau(X)^2 \cdot (\tau(k) \cdot \tau(Y) + 1))^2 \cdot \tau(Z) \\
 &= \tau(X)^4 \cdot (\tau(k) \cdot \tau(Y) + 1)^2 \cdot \tau(Z) \\
 &= \tau(X)^4 \cdot (\tau(k)^2 \cdot \tau(Y)^2 + 2 \cdot \tau(k) \cdot \tau(Y) + 1) \cdot \tau(Z) \\
 &= \tau(X)^4 \cdot \tau(k)^2 \cdot \tau(Y)^2 \cdot \tau(Z) + \tau(X)^4 \cdot 2 \cdot \tau(k) \cdot \tau(Y) \cdot \tau(Z) + \tau(X)^4 \cdot \tau(Z)
 \end{aligned}$$

$$\begin{aligned}
 \tau(r) &= \tau(k(X \otimes Y \otimes Z)) \\
 &= \tau(k) \cdot (\tau(X)^2 \cdot \tau(Y))^2 \cdot \tau(Z) + 1 \\
 &= \tau(k) \cdot \tau(X)^4 \cdot \tau(Y)^2 \cdot \tau(Z) + 1
 \end{aligned}$$

Then, since all nonnegative integer interpretation are greater or equal to two, $\tau(X)^4 \cdot \tau(k)^2 \cdot \tau(Y)^2 \cdot \tau(Z)$ is clearly greater than $\tau(k) \cdot \tau(X)^4 \cdot \tau(Y)^2 \cdot \tau(Z)$, $\tau(X)^4 \cdot \tau(Z)$ is greater than 1, and thus $\tau(l) - \tau(r)$ is greater than 0.

For the second one:

$$\begin{aligned}
 \tau(l) &= \tau(X \otimes (Y + Z) \otimes W) \\
 &= (\tau(X)^2 \cdot (\tau(Y) + \tau(Z)))^2 \cdot \tau(W) \\
 &= (\tau(X)^4 \cdot (\tau(Y)^2 + 2\tau(Y) \cdot \tau(Z) + \tau(Z)^2) \cdot \tau(W) \\
 &= \tau(X)^4 \cdot \tau(Y)^2 \cdot \tau(W) + \tau(X)^4 \cdot 2\tau(Y) \cdot \tau(Z) \cdot \tau(W) + \tau(X)^4 \cdot \tau(Z)^2 \cdot \tau(W)
 \end{aligned}$$

$$\begin{aligned}
 \tau(r) &= \tau(X \otimes Y \otimes W + X \otimes Z \otimes W) \\
 &= \tau(X \otimes Y \otimes W) + \tau(X \otimes Z \otimes W) \\
 &= (\tau(X)^2 \cdot \tau(Y))^2 \cdot \tau(W) + (\tau(X)^2 \cdot \tau(Z))^2 \cdot \tau(W) \\
 &= \tau(X)^4 \cdot \tau(Y)^2 \cdot \tau(W) + \tau(X)^4 \cdot \tau(Z)^2 \cdot \tau(W)
 \end{aligned}$$

Then, $\tau(l) - \tau(r) = \tau(X)^4 \cdot 2\tau(Y) \cdot \tau(Z) \cdot \tau(W)$; which, since $\tau(0) = 2$ and all other interpretations are positive, is also positive for all nonnegative integer interpretations.

For the third one:

$$\begin{aligned}
 \tau(l) &= \tau(X \times (Y + Z) \times W) \\
 &= \tau(X) \cdot (\tau(Y) + \tau(Z)) \cdot \tau(X) \cdot \tau(W) \cdot \tau(X) \cdot (\tau(Y) + \tau(Z)) \cdot \tau(X) \\
 &= \tau(X) \cdot \tau(Y) \cdot \tau(X) \cdot \tau(W) \cdot \tau(X) \cdot \tau(Y) \cdot \tau(X) \\
 &\quad + \tau(X) \cdot \tau(Y) \cdot \tau(X) \cdot \tau(W) \cdot \tau(X) \cdot \tau(Z) \cdot \tau(X) \\
 &\quad + \tau(X) \cdot \tau(Z) \cdot \tau(X) \cdot \tau(W) \cdot \tau(X) \cdot \tau(Y) \cdot \tau(X) \\
 &\quad + \tau(X) \cdot \tau(Z) \cdot \tau(X) \cdot \tau(W) \cdot \tau(X) \cdot \tau(Z) \cdot \tau(X)
 \end{aligned}$$

$$\begin{aligned}
 \tau(r) &= \tau(X \times Y \times W + X \times Z \times W) \\
 &= \tau(X) \cdot \tau(Y) \cdot \tau(X) \cdot \tau(W) \cdot \tau(X) \cdot \tau(Y) \cdot \tau(X) \\
 &\quad + \tau(X) \cdot \tau(Z) \cdot \tau(X) \cdot \tau(W) \cdot \tau(X) \cdot \tau(Z) \cdot \tau(X)
 \end{aligned}$$

$$\begin{aligned}
 \tau(l) - \tau(r) &= \tau(X) \cdot \tau(Y) \cdot \tau(X) \cdot \tau(W) \cdot \tau(X) \cdot \tau(Z) \cdot \tau(X) \\
 &\quad + \tau(X) \cdot \tau(Z) \cdot \tau(X) \cdot \tau(W) \cdot \tau(X) \cdot \tau(Y) \cdot \tau(X)
 \end{aligned}$$

Which again, since all nonnegative integers interpretations (including $\tau(0) = 2$) are positive, is always positive.

Finally, for the fourth one:

$$\begin{aligned}\tau(l) &= \tau(X + kY + Z + k2Y + W) \\ &= \tau(X) + (\tau(k) \cdot \tau(Y) + 1) + \tau(Z) + (\tau(k2) \cdot \tau(Y) + 1) + \tau(W) \\ &= \tau(k) \cdot \tau(Y) + \tau(k2) \cdot \tau(Y) + \tau(X) + \tau(Z) + \tau(W) + 2\end{aligned}$$

$$\begin{aligned}\tau(r) &= \tau(X + (k + k2)Y + Z + W) \\ &= \tau(X) + ((\tau(k) + \tau(k2)) \cdot \tau(Y) + 1) + \tau(Z) + \tau(W) \\ &= \tau(X) + \tau(k) \cdot \tau(Y) + \tau(k2) \cdot \tau(Y) + 1 + \tau(Z) + \tau(W) \\ &= \tau(k) \cdot \tau(Y) + \tau(k2) \cdot \tau(Y) + \tau(X) + \tau(Z) + \tau(W) + 1\end{aligned}$$

Then, $\tau(l) - \tau(r) = 1$, a positive constant.

It was demonstrated that, using the polynomial interpretation defined above, for all rules of our term-rewriting system $\tau(l) - \tau(r)$ were positive.

$\therefore$ All rules are reducing

$\therefore$ The term-rewriting system used is terminating.